

Guía Práctica de SQL — De Cero a Portafolio

Preparado para Matías González Ferreyra

Agosto 2026

Contents

Introducción	2
Herramientas que vas a necesitar	2
Etapla 1: Conceptos fundamentales de bases de datos	2
Teoría	2
Ejemplo: crear tu primera tabla	3
Tarea / Proyecto: “Diseña tu primera base de datos”	3
Etapla 2: Consultas básicas (SELECT)	3
Teoría	3
Ejemplos	3
Tarea / Proyecto: “Consultas sobre tu base de datos”	4
Etapla 3: Filtrado avanzado	4
Teoría	4
Ejemplos	4
Tarea / Proyecto: “Filtros complejos”	5
Etapla 4: Funciones de agregación y agrupación	5
Teoría	5
Ejemplos	5
Tarea / Proyecto: “Reporte de nómina agrupado”	5
Etapla 5: Relacionar tablas (JOIN)	6
Teoría	6
Ejemplo con dos tablas nuevas	6
Tarea / Proyecto: “Base de datos relacional de una empresa ficticia”	6
Etapla 6: Subconsultas	7
Teoría	7
Ejemplos	7
Tarea / Proyecto: “Preguntas de negocio con subconsultas”	7
Etapla 7: Modificar datos y estructura (INSERT, UPDATE, DELETE, ALTER)	7
Teoría	7
Ejemplos	7
Tarea / Proyecto: “Simulación de mantenimiento de base de datos”	8
Proyecto final integrador: “Base de datos de análisis industrial”	8
Recursos recomendados	8

Introducción

SQL (Structured Query Language) es el lenguaje que se usa para hablar con bases de datos: pedirles información, filtrarla, combinarla y modificarla. Es una de las habilidades más pedidas en análisis de datos, y complementa directamente tu experiencia con Excel: piensa en SQL como “Excel a gran escala”, donde en vez de hojas de cálculo tienes tablas relacionadas entre sí.

Cada etapa de esta guía tiene:

1. **Teoría** en palabras simples.
2. **Ejemplos** de consultas listas para ejecutar.
3. **Tarea o mini-proyecto** que puedes subir a GitHub como evidencia de práctica.

Herramientas que vas a necesitar

Herramienta	Para qué sirve	Dónde conseguirla
SQLite	Motor de base de datos ligero, no necesita instalación de servidor	Ya viene incluido en la mayoría de sistemas; también sqlite.org
DB Browser for SQLite	Interfaz visual para crear y consultar bases de datos SQLite	sqlitebrowser.org
Extensión “SQLite” o “SQLTools” en VS Code	Ejecutar consultas SQL desde tu editor	Marketplace de VS Code
(Opcional, para practicar en la nube) PostgreSQL + pgAdmin	Motor más usado en la industria, útil una vez domines lo básico	postgresql.org

Para empezar, usa **SQLite + DB Browser for SQLite**: es gratis, no requiere instalar un servidor, y es perfecto para aprender. Cuando domines lo básico, migrar a PostgreSQL será sencillo porque el lenguaje SQL es prácticamente el mismo.

Etapas: Conceptos fundamentales de bases de datos

Teoría

Una **base de datos** es un conjunto organizado de **tablas**. Cada tabla es como una hoja de Excel:

- Una **fila** (registro) es una entrada individual (ej. un empleado).
- Una **columna** (campo) es un atributo compartido por todas las filas (ej. nombre, edad, salario).
- Cada columna tiene un **tipo de dato**: INTEGER (números enteros), TEXT (texto), REAL (decimales), DATE (fechas).
- La **clave primaria** (PRIMARY KEY) es una columna que identifica de forma única cada fila (como el número de empleado, que nunca se repite).

- La **clave foránea** (FOREIGN KEY) conecta una tabla con otra (ej. la tabla “Pedidos” referencia a la tabla “Clientes”).

Piensa en una base de datos como un archivero de oficina: cada cajón es una tabla, cada carpeta dentro es una fila, y cada pestaña de información dentro de la carpeta es una columna.

Ejemplo: crear tu primera tabla

```
CREATE TABLE empleados (
  id INTEGER PRIMARY KEY,
  nombre TEXT NOT NULL,
  puesto TEXT,
  salario REAL,
  fecha_ingreso DATE
);
```

Tarea / Proyecto: “Diseña tu primera base de datos”

Usando DB Browser for SQLite, crea una base de datos llamada empresa.db con una tabla empleados como la del ejemplo. Agrega manualmente 5 filas de datos ficticios (o basados en tu experiencia real en Quaker Houghton, sin datos confidenciales). Guarda el archivo .db y súbelo a un repositorio nuevo en GitHub llamado sql-practica, junto con un README.md explicando qué representa la tabla.

Etapas 2: Consultas básicas (SELECT)

Teoría

SELECT es el comando más usado en SQL: sirve para **leer** datos de una tabla.

- SELECT column1, column2 FROM tabla; — elige columnas específicas.
- SELECT * FROM tabla; — el asterisco significa “todas las columnas”.
- WHERE — filtra filas según una condición.
- ORDER BY — ordena los resultados (agrega DESC para orden descendente).
- LIMIT — restringe cuántas filas se muestran.

Ejemplos

```
-- Ver todos los empleados
SELECT * FROM empleados;

-- Ver solo nombre y salario
SELECT nombre, salario FROM empleados;

-- Filtrar: empleados con salario mayor a 15000
SELECT nombre, salario FROM empleados
WHERE salario > 15000;

-- Ordenar de mayor a menor salario
SELECT nombre, salario FROM empleados
ORDER BY salario DESC;
```

```
-- Mostrar solo los 3 empleados con más salario
SELECT nombre, salario FROM empleados
ORDER BY salario DESC
LIMIT 3;
```

Tarea / Proyecto: “Consultas sobre tu base de datos”

Sobre la tabla empleados que creaste en la Etapa 1, escribe y guarda (en un archivo consultas_etapa2.sql) al menos 5 consultas distintas: mostrar todos los empleados ordenados por fecha de ingreso, mostrar solo los que ganan más de cierto monto, mostrar los 2 empleados más recientes, etc. Súbelo a tu repositorio sql-practica.

Etapa 3: Filtrado avanzado

Teoría

Además de WHERE con comparaciones simples, existen operadores para filtros más específicos:

- AND / OR — combinar varias condiciones.
- IN (valor1, valor2, ...) — verifica si un valor está dentro de una lista.
- BETWEEN valor1 AND valor2 — verifica si un valor está en un rango.
- LIKE 'patron%' — busca coincidencias de texto parcial (% significa “cualquier cosa”).
- IS NULL / IS NOT NULL — verifica si un campo está vacío o no.

Ejemplos

```
-- AND: empleados con salario alto Y puesto específico
SELECT nombre FROM empleados
WHERE salario > 15000 AND puesto = 'Analista';

-- OR: empleados en dos puestos distintos
SELECT nombre FROM empleados
WHERE puesto = 'Analista' OR puesto = 'Técnico de Campo';

-- IN: lo mismo que el OR anterior, más limpio
SELECT nombre FROM empleados
WHERE puesto IN ('Analista', 'Técnico de Campo');

-- BETWEEN: salarios en un rango
SELECT nombre, salario FROM empleados
WHERE salario BETWEEN 14000 AND 18000;

-- LIKE: nombres que empiezan con "M"
SELECT nombre FROM empleados
WHERE nombre LIKE 'M%';

-- IS NULL: empleados sin fecha de ingreso registrada
SELECT nombre FROM empleados
WHERE fecha_ingreso IS NULL;
```

Tarea / Proyecto: “Filtros complejos”

Agrega al menos 10 empleados más a tu tabla (puedes generarlos con datos ficticios variados en puesto y salario). Escribe 5 consultas usando AND, OR, IN, BETWEEN y LIKE respectivamente. Guarda todo en consultas_etapa3.sql y súbelo.

Etapa 4: Funciones de agregación y agrupación

Teoría

Cuando quieres resumir datos (no ver fila por fila, sino totales o promedios), usas funciones de agregación:

- COUNT() — cuenta filas.
- SUM() — suma valores.
- AVG() — promedio.
- MAX() / MIN() — valor máximo/mínimo.
- GROUP BY — agrupa filas que comparten un valor en común, para aplicar las funciones anteriores a cada grupo por separado.
- HAVING — como WHERE, pero para filtrar *después* de agrupar (porque WHERE no puede filtrar sobre resultados agregados).

Piensa en GROUP BY como hacer una tabla dinámica en Excel: agrupas por una columna (ej. puesto) y calculas un resumen (ej. salario promedio) para cada grupo.

Ejemplos

```
-- Cuántos empleados hay en total
SELECT COUNT(*) FROM empleados;

-- Salario promedio de todos los empleados
SELECT AVG(salario) FROM empleados;

-- Salario total y promedio por puesto
SELECT puesto, COUNT(*) AS cantidad, AVG(salario) AS salario_promedio
FROM empleados
GROUP BY puesto;

-- Solo mostrar puestos donde hay más de 2 empleados
SELECT puesto, COUNT(*) AS cantidad
FROM empleados
GROUP BY puesto
HAVING COUNT(*) > 2;
```

Tarea / Proyecto: “Reporte de nómina agrupado”

Escribe consultas que respondan (guárdalas en consultas_etapa4.sql):

1. ¿Cuántos empleados hay por puesto?
2. ¿Cuál es el salario promedio, máximo y mínimo por puesto?
3. ¿Qué puestos tienen un salario promedio superior a un monto que definas?

Este tipo de reporte es exactamente lo que se pide en entrevistas de analista de datos junior.

Etapa 5: Relacionar tablas (JOIN)

Teoría

En bases de datos reales, la información no vive en una sola tabla — vive repartida en varias tablas relacionadas, para evitar repetir datos. Los JOIN combinan filas de dos o más tablas basándose en una columna en común (normalmente clave primaria/foránea).

- INNER JOIN — muestra solo las filas que tienen coincidencia en ambas tablas.
- LEFT JOIN — muestra todas las filas de la tabla izquierda, aunque no tengan coincidencia en la derecha (rellena con NULL).
- RIGHT JOIN — lo mismo pero al revés (SQLite no lo soporta directamente, pero PostgreSQL sí).

Piensa en dos hojas de Excel: una con clientes y otra con pedidos. Un JOIN es como usar BUSCARV pero para combinar filas completas, no solo un valor.

Ejemplo con dos tablas nuevas

```
CREATE TABLE departamentos (  
  id INTEGER PRIMARY KEY,  
  nombre TEXT  
);  
  
CREATE TABLE empleados_v2 (  
  id INTEGER PRIMARY KEY,  
  nombre TEXT,  
  departamento_id INTEGER,  
  FOREIGN KEY (departamento_id) REFERENCES departamentos(id)  
);  
  
-- INNER JOIN: empleados con su departamento  
SELECT empleados_v2.nombre, departamentos.nombre AS departamento  
FROM empleados_v2  
INNER JOIN departamentos ON empleados_v2.departamento_id = departamentos.id;  
  
-- LEFT JOIN: todos los empleados, incluso sin departamento asignado  
SELECT empleados_v2.nombre, departamentos.nombre AS departamento  
FROM empleados_v2  
LEFT JOIN departamentos ON empleados_v2.departamento_id = departamentos.id;
```

Tarea / Proyecto: “Base de datos relacional de una empresa ficticia”

Diseña una base de datos con al menos 3 tablas relacionadas entre sí (por ejemplo: clientes, productos, pedidos, donde cada pedido tiene un cliente y un producto). Llena cada tabla con al menos 10 filas de datos ficticios. Escribe 5 consultas con JOIN que respondan preguntas de negocio (ej. “¿qué cliente ha comprado más?”, “¿qué producto se vende más?”). Súbelo como un proyecto nuevo y completo en tu repositorio sql-practica, con su propio README.md explicando el diseño de las tablas (puedes incluir un diagrama simple).

Etapa 6: Subconsultas

Teoría

Una **subconsulta** es una consulta SELECT dentro de otra consulta. Se usa cuando necesitas el resultado de una consulta para filtrar o comparar en otra.

Piensa en una subconsulta como una pregunta dentro de otra pregunta: “muéstrame los empleados que ganan más que el promedio” primero necesita calcular el promedio (una consulta), y luego comparar cada salario contra ese resultado (otra consulta).

Ejemplos

```
-- Empleados que ganan más que el promedio general
SELECT nombre, salario FROM empleados
WHERE salario > (SELECT AVG(salario) FROM empleados);

-- Departamentos que tienen al menos un empleado
SELECT nombre FROM departamentos
WHERE id IN (SELECT DISTINCT departamento_id FROM empleados_v2);

-- El empleado con el salario más alto
SELECT nombre, salario FROM empleados
WHERE salario = (SELECT MAX(salario) FROM empleados);
```

Tarea / Proyecto: “Preguntas de negocio con subconsultas”

Sobre tu base de datos relacional de la Etapa 5, escribe 4 consultas que usen subconsultas para responder preguntas como: “¿qué productos nunca se han vendido?”, “¿qué cliente compró el pedido más caro?”. Guarda las consultas en consultas_etapa6.sql.

Etapa 7: Modificar datos y estructura (INSERT, UPDATE, DELETE, ALTER)

Teoría

Hasta ahora solo has leído datos. SQL también permite **escribir** y **modificar**:

- INSERT INTO tabla (columnas) VALUES (valores); — agrega una fila nueva.
- UPDATE tabla SET columna = valor WHERE condicion; — modifica filas existentes. **Cuidado:** si olvidas el WHERE, se actualizan TODAS las filas.
- DELETE FROM tabla WHERE condicion; — elimina filas. Mismo cuidado con el WHERE.
- ALTER TABLE tabla ADD COLUMN columna tipo; — agrega una columna nueva a una tabla ya existente.

Ejemplos

```
-- Insertar un nuevo empleado
INSERT INTO empleados (nombre, puesto, salario, fecha_ingreso)
VALUES ('Laura Ruiz', 'Analista', 16500, '2026-08-01');
```

```
-- Actualizar el salario de un empleado específico
UPDATE empleados
SET salario = 17000
WHERE nombre = 'Laura Ruiz';

-- Eliminar un empleado específico
DELETE FROM empleados
WHERE nombre = 'Laura Ruiz';

-- Agregar una columna nueva a la tabla
ALTER TABLE empleados ADD COLUMN correo TEXT;
```

Tarea / Proyecto: “Simulación de mantenimiento de base de datos”

Sobre tu base de datos de la Etapa 5, simula un ciclo real de mantenimiento: agrega 3 registros nuevos, actualiza 2 registros existentes, elimina 1 registro, y agrega una columna nueva a una de las tablas. Documenta cada paso con comentarios en un archivo `mantenimiento.sql` y súbelo.

Proyecto final integrador: “Base de datos de análisis industrial”

Como conexión directa con tu experiencia real en Quaker Houghton, diseña una base de datos que simule el monitoreo de parámetros químicos que ya manejas en tu trabajo:

1. Tabla equipos (id, nombre, ubicación).
2. Tabla mediciones (id, equipo_id, fecha, parametro, valor) relacionada con equipos.
3. Al menos 30 mediciones ficticias, distribuidas en varios equipos y fechas.
4. Consultas que respondan: ¿qué equipo tiene más mediciones fuera de un rango normal que defines? ¿cuál es el promedio de cada parámetro por equipo? ¿qué equipo no ha sido medido en los últimos X días?

Este proyecto es el más valioso de todos para tu portafolio, porque conecta tu experiencia laboral real con una habilidad técnica nueva — es exactamente el tipo de historia que se cuenta bien en una entrevista.

Sube el proyecto completo con: - El archivo `.db` o el script `.sql` que crea las tablas. - Un archivo con las consultas y una breve explicación de qué responde cada una. - Un `README.md` explicando el contexto (sin usar datos confidenciales reales de la empresa).

Recursos recomendados

- **Práctica interactiva gratuita:** SQLZoo (sqlzoo.net) y Mode SQL Tutorial (mode.com/sql-tutorial).
- **Documentación oficial de SQLite:** sqlite.org/lang.html
- **Siguiente paso natural:** una vez domines esto, aprende a conectar SQL con Python usando la librería `sqlite3` (incluida en Python) para combinar ambas habilidades en un solo proyecto de análisis de datos.

Cómo subir cada proyecto a GitHub

```
cd carpeta_del_proyecto
git init
git add .
git commit -m "Primer commit: agregó base de datos y consultas de la etapa X"
git branch -M main
git remote add origin https://github.com/MatiasGonzalez-Dev/sql-practica.git
git push -u origin main
```